

Complexity

What is the central question? What do I aim to achieve; aim to analyse?

The question is: “Can an interpreter reach JIT-like performance without JIT-like complexity?”

The key detail in the question is the comparative nature of it. It has less hinging on the absolute complexities of Binary Translating Interpreter(BTI)s but rather focuses on the relative complexities to other interpreters and JIT compilers. This is the key argument for why specific measures were chosen in the following sections.

In the interpreter space there are some established constructs and techniques. In traditional software engineering these techniques are seen as esoteric and complex. For the purposed of analysing binary translators these techniques are going to be considered as elementary. These constructs may well reduce the complexity due to their decoumented nature and familiarity of developers in the problem space.

List of binary translating interpreter constructs and techniques

The expectation is that the main factor to the complication in a binary translator will be the composition of the established techniques. Really the complication is the amount of “glue” required to use these techniques.

What are the available metrics that answer or partially answer the question? Why do they answer that proportion and what are the implications of picking them? Are there any domain/problem specific metrics I can pick? Create new metrics?

There are a number of standard metrics, Cyclometric, fan-in & fan-out, etc. The metrics offer an insight into the general complexity of the code. It is a measure of the number of decision points, operations and syntatic complexity.

The problem with using just standard metrics is, as outlined above, that the metrics are un-specific to the problem domain of binary translating interpreters. They offer either too much insight obfuscating the “real” complexity in any given interpreter design, or they don’t offer enough visablility into the truly complex areas of an interpreter.

Really standardised metrics and domain specific metrics must be used in tandem, one highlighting the general complexity statistics allowing for comparison with systems that are not binary translators, and the domain specific metrics highlighting statistics that allow for indepth comparisons between binary translators.

A crucial factor to consider when designing a domain specific metric is the demographic. Using domain specific metrics will limit the accessibility for general developers as there will be an expected level of expertise required to understand the complexity comparisons. A mitigation of this issue could be the use of both kinds of metrics. Using both metrics standard and domain specific, we could equate complexity “ratios” in domain specific metrics with standardised ones showing that they both track each other.

To gather the points together and answer the questions in bold for each metric:

STANDARD

Cyclometric Complexity | Partial | Gives equal importance to each decision/construct doesn’t take into account standard procedures and techniques specific to BTIs

Fan-In & Fan-Out | Partial | Expect it to be quite low in general due to the nature of interpreters. E.g in computed-gotos there are not many instructions but the complexity is still fairly high. Though an interesting point is that the amount of “glue” code should be quantifiable using this metric.

Single Lines of Code | Partial | Simplistic, doesn’t take into account the content of the lines.

Algorithmic Complexity | Complete | Difficult, but allows for a thorough analysis of the components of each interpreter. Standard techniques can be “abstracted” away allowing for a fixed cost to be applied.

UNIQUE/DOMAIN SPECIFIC

Modules/Components | Partial | If all components are given the same “value”, designs with few but complex components will be less complex. Must be paired with other metrics to design a heuristic that allows for complexity based weighting giving a better overview.

Binary Translating Interpreter(BTI) standard techniques | Partial | This is an effective strategy, it makes the complexity specific to the problem domain, whilst taking into account the expected knowledge when developing BTIs. The method would be to create a complexity heuristic for each technique. It is best paired with the algorithmic analysis metric as it will allow for quantification of “glue” code as well.

Which metrics have I decided on picking? What makes those metrics suitable for the questions I am asking? Is there any affect on my choice due to the problem itself? Is it based off convenience? Maybe other studies have used those metrics? How does choosing these metrics shape the study?

Are there changes/re-interpretations of these metrics that need to be made? Will changing them subtly make them more applicable?

How will I measure these metrics? What are the methods, programs, tools and techniques?

How will I compare the metrics, what is considered “good” and what is considered “bad”? Is there a baseline or reference point?